

گزارش نهایی پروژه Wumpus World

نام دانشجو: ماهان ورمزیار شماره دانشجویی: 403243106 نام درس: هوش مصنوعی نام استاد: استاد شمس فرد

چکیده

در این پروژه، محیط کلاسیک Wumpus World روی یک گرید 8×8 پیاده‌سازی شده و سه رویکرد متفاوت برای حل آن با یکدیگر مقایسه شده‌اند:

1. عامل A-Star با دسترسی کامل به نقشه؛

2. عامل آنلاین Rule-Based مبتنی بر ادراک‌های محلی، پایگاه دانش و backtracking؛

3. عامل آنلاین Hybrid Genetic که تصمیم‌های مرحله اکتشاف را با وزن‌های بهینه‌شده توسط الگوریتم ژنتیک می‌گیرد.

هدف عامل این است که از خانه (1,1) حرکت را آغاز کند، حداقل یک طلا جمع‌آوری کند و پیش از مرگ یا تمام شدن جان، به خانه خروج برسد. محیط شامل دیوار، چاه، گول، طلا و خروج است. عامل‌های آنلاین نقشه واقعی را مشاهده نمی‌کنند و فقط از اطلاعاتی مانند موقعیت فعلی، جان، Breeze، Stench، وجود چاه در خانه فعلی، حرکت‌های معتبر و حافظه خانه‌های دیده‌شده استفاده می‌کنند.

برای آموزش عامل ژنتیکی، ۱۲ نقشه آموزشی به کار رفته است. ارزیابی نهایی روی ۳۰ نقشه مستقل شامل ۱۰ نقشه آسان، ۱۰ نقشه متوسط و ۱۰ نقشه سخت انجام شده است. نتایج نشان داد A-Star به دلیل داشتن اطلاعات کامل نقشه، در همه اجراها موفق بود. در میان دو عامل آنلاین، عامل قاعده‌محور نرخ موفقیت بیشتری داشت، اما عامل ژنتیکی ترکیبی در اپیزودهای موفق با تعداد حرکت کمتری به هدف رسید.

روش	نرخ موفقیت	امتیاز متوسط	حرکت متوسط همه اجراها	حرکت متوسط اجرای موفق
A-Star	100.00%	157.60	12.40	12.40
Rule-Based	90.00%	117.93	32.90	32.30
Hybrid Genetic	83.33%	120.97	31.80	24.60

۱. تعریف مسئله و هدف پروژه

۱.۱ مسئله Wumpus World

Wumpus World یک محیط کلاسیک در هوش مصنوعی است که برای بررسی موضوعاتی مانند جست و جو، استنتاج منطقی، تصمیم‌گیری در شرایط عدم قطعیت و یادگیری استفاده می‌شود. محیط از تعدادی خانه تشکیل شده است و عامل باید با اطلاعات محدود در آن حرکت کند.

در پیاده‌سازی این پروژه، گرید دارای ۸ سطر و ۸ ستون است. عامل همیشه از خانه داخلی $(0,0)$ ، معادل $(1,1)$ در نمایش کاربر، شروع می‌کند. نمادهای نقشه عبارت‌اند از:

نماد	معنی
*	خانه خالی
D	دیوار
P	چاه
W	غول یا Wumpus
G	طلا

مختصات خروج در چهار خط تنظیمات انتهای فایل نقشه مشخص می‌شود.

۱.۲ هدف عامل

عامل زمانی موفق محسوب می‌شود که:

۱. حداقل یک طلا جمع‌آوری کرده باشد؛

۲. زنده بماند؛

۳. به مختصات خروج برسد.

عامل می‌تواند در هر مرحله یکی از چهار عمل زیر را انتخاب کند:

UP, DOWN, LEFT, RIGHT

۱.۳ دلیل مقایسه سه روش

سه روش پروژه از نظر سطح اطلاعات و نوع تصمیم‌گیری متفاوت‌اند:

- A-Star** کل نقشه را مشاهده می‌کند و بهترین baseline ممکن برای محیط شناخته‌شده را ارائه می‌دهد.
- Rule-Based** نقشه را نمی‌بیند و با قواعد منطقی و حافظه، خانه‌های امن و خطرناک را تخمین می‌زند.
- Hybrid Genetic** همان اطلاعات محلی عامل قاعده‌محور را دریافت می‌کند، اما اهمیت ویژگی‌های تصمیم‌گیری را به جای تنظیم دستی، از طریق الگوریتم ژنتیک یاد می‌گیرد.

به همین دلیل، مقایسه مستقیم A-Star با دو روش آنلاین کاملاً هم‌سطح نیست. A-Star در این پروژه نقش **Oracle** یا کران بالای عملکرد را دارد و مقایسه اصلی و منصفانه‌تر میان Rule-Based و Hybrid Genetic انجام می‌شود.

۲. معماری کلی سیستم

پروژه به صورت ماژولار طراحی شده است تا محیط، عامل‌ها، آموزش و آزمایش مستقل از یکدیگر باشند.

ماژول	نقش اصلی
map_parser.py	خواندن و اعتبارسنجی فایل نقشه
environment.py	اجرای قوانین بازی، تولید observation و محاسبه امتیاز
base_agent.py	تعریف رابط مشترک عامل‌ها
astar_agent.py	پیاده‌سازی A-Star با اطلاعات کامل
knowledge_base.py	نگهداری دانش و انجام استنتاج محلی
rule_based_agent.py	تصمیم‌گیری قاعده‌محور و backtracking
genetic_agent.py	سیاست وزن‌دار عامل ژنتیکی ترکیبی
genetic_algorithm.py	آموزش وزن‌ها با الگوریتم ژنتیک
map_generator.py	تولید نقشه‌های آموزش و تست با seed ثابت
main.py	اجرای یک اپیزود با عامل انتخاب‌شده
experiment.py	اجرای آزمایش نهایی، ذخیره CSV و رسم نمودارها

جریان کلی اجرای یک اپیزود به شکل زیر است:

flowchart TD

```
A[خواندن فایل نقشه] --> B[اعتبارسنجی توسط map_parser]
B --> C[WumpusEnvironment ساخت]
C --> D[ساخت عامل انتخاب‌شده]
D --> E[محیط و عامل reset]
E --> F[محلی observation دریافت]
F --> G[توسط عامل action انتخاب]
G --> H[environment در action اجرای]
H --> I[اپیزود تمام شده؟]
I -->|خیر| F
I -->|بله| J[ثبت نتیجه و معیارها]
```

کلاس **BaseAgent** یک قرارداد مشترک برای عامل‌ها ایجاد می‌کند. هر عامل باید حافظه خود را با **reset()** پاک کند و با **choose_action()** از روی observation یک حرکت برگرداند. به این ترتیب **main.py** می‌تواند بدون وابستگی به جزئیات داخلی، هر سه روش را در همان حلقه اجرا کند.

۳. محیط شبیه‌سازی و قوانین بازی

۳.۱ وضعیت بازی

در `environment.py` وضعیت جاری بازی با کلاس `GameState` نگهداری می‌شود. مهم‌ترین فیلدهای آن عبارت‌اند از:

```
@dataclass
class GameState:
    position: tuple[int, int] = (0, 0)
    health: int = 0
    collected_gold: int = 0
    pit_entries: int = 0
    steps: int = 0
    score: int = 0
    success: bool = False
    done: bool = False
    termination_reason: str | None = None
```

علاوه بر این موارد، خانه‌های بازدیدشده و تاریخچه حرکت‌ها نیز ذخیره می‌شوند.

۳.۲ ادراک‌های عامل

تابع `observe()` اطلاعات مجاز را به عامل برمی‌گرداند. مهم‌ترین فیلدها عبارت‌اند از:

- `position`: مختصات فعلی؛
- `health`: جان باقی‌مانده؛
- `breeze`: وجود چاه در یکی از همسایه‌های چهارجهته؛
- `stench`: وجود غول در یکی از همسایه‌های چهارجهته؛
- `pit_here`: قرار داشتن عامل در چاه؛
- `gold_here`: قرار داشتن طلا در خانه فعلی؛
- `has_gold`: جمع‌آوری شدن حداقل یک طلا؛
- `at_exit`: قرار داشتن روی خروج؛
- `valid_actions`: حرکت‌هایی که از محدوده خارج نمی‌شوند و به دیوار برخورد نمی‌کنند.

دو عامل آنلاین فقط این `observation` را دریافت می‌کنند و به `grid` مخفی نقشه دسترسی مستقیم ندارند.

۳.۳ اثر خانه‌های مختلف

- هر تلاش برای حرکت، حتی حرکت مسدودشده، یک واحد جان کم می‌کند.
- حرکت خارج از محدوده یا ورود به دیوار باعث جابه‌جایی عامل نمی‌شود.
- ورود به چاه ابتدا هزینه یک حرکت را کم می‌کند و سپس جان را با تقسیم صحیح بر دو نصف می‌کند.
- ورود به خانه غول، جان را صفر کرده و ایزود را فوراً پایان می‌دهد.
- ورود به خانه طلا باعث جمع‌آوری آن می‌شود.
- ورود به خروج بدون طلا، شکست با علت `escaped_without_gold` است.

- ورود به خروج همراه طلا، موفقیت با علت `escaped_with_gold` است.

۳.۴ امتیاز و Reward

امتیاز محیط با فرمول زیر محاسبه می‌شود:

```
score = health + collected_gold * gold_score - pit_entries * pit_penalty
```

مقادیر `gold_score` و `pit_penalty` از فایل نقشه خوانده می‌شوند. در نقشه‌های تولیدشده پروژه، این مقادیر به ترتیب ۵۰ و ۱۰ هستند.

Reward هر حرکت، تغییر امتیاز نسبت به مرحله قبل است:

```
self.state.score = self._calculate_score()
reward = self.state.score - old_score
```

این طراحی باعث می‌شود کاهش جان، جمع‌آوری طلا و ورود به چاه مستقیماً در Reward منعکس شوند.

۳.۵ شرایط پایان

اپیزود در یکی از حالت‌های زیر تمام می‌شود:

علت	نتیجه
<code>escaped_with_gold</code>	موفقیت
<code>escaped_without_gold</code>	شکست
<code>wumpus</code>	شکست و مرگ فوری
<code>health_depleted</code>	شکست به علت تمام شدن جان
<code>max_steps</code>	شکست به علت رسیدن به سقف حرکت
<code>agent_stopped</code>	توقف عامل به دلیل نداشتن تصمیم معتبر

۴. روش اول: عامل A-Star

۴.۱ ایده اصلی الگوریتم

A-Star یک الگوریتم جست‌وجوی آگاهانه است که هر حالت را با مجموع دو مقدار ارزیابی می‌کند:

$$f(n) = g(n) + h(n)$$

- $g(n)$ هزینه واقعی مسیر از شروع تا حالت n است.
- $h(n)$ تخمینی از هزینه باقی‌مانده تا هدف است.

حالت جست‌وجو در پروژه فقط شامل سه مؤلفه است:

```
@dataclass(frozen=True)
class SearchState:
    position: tuple[int, int]
    health: int
    has_gold: bool
```

بنابراین جهت نگاه عامل، تیر و وضعیت زنده یا مرده بودن غول در مدل وجود ندارند. غول‌ها در A-Star به‌صورت خانه غیرقابل‌عبور در نظر گرفته می‌شوند.

۴.۲ هدف جست‌وجو

شرط هدف زمانی برقرار است که عامل:

- طلا داشته باشد؛
- روی خانه خروج قرار گرفته باشد.

این شرط در حلقه جست‌وجو به شکل زیر بررسی می‌شود:

```
if current.has_gold and current.position == self.config.exit_position:
    actions, path = self._reconstruct(came_from, current)
    return PlanResult(
        actions=actions,
        path=path,
        total_cost=current_cost,
        final_health=current.health,
        expanded_nodes=expanded_nodes,
    )
```

۴.۳ تولید حالت‌های جانشین

برای هر حالت، چهار جهت اصلی بررسی می‌شوند. حالات زیر حذف می‌شوند:

- خارج از محدوده؛
- دیوار؛
- غول؛
- خروج قبل از گرفتن طلا؛
- حالتی که جان عامل در آن صفر یا منفی شود.

چاه قابل عبور است، اما هزینه بیشتری ایجاد می‌کند. کد اصلی محاسبه هزینه انتقال چنین است:

```

next_health = state.health - 1
pit_cost = 0
if cell == "P":
    next_health //= 2
    pit_cost = self.config.pit_penalty
if next_health <= 0:
    continue

health_loss = state.health - next_health
transition_cost = health_loss + pit_cost

```

در نتیجه A-Star فقط تعداد قدم‌ها را کمینه نمی‌کند؛ بلکه کاهش واقعی جان و جریمه چاه را نیز در هزینه مسیر وارد می‌کند. بنابراین ممکن است یک مسیر کمی طولانی‌تر اما امن‌تر را به مسیر کوتاه دارای چاه ترجیح دهد.

۴.۴ تابع Heuristic

اگر عامل هنوز طلا نگرفته باشد، تخمین فاصله برابر است با کمترین مجموع فاصله عامل تا یکی از طلاها و فاصله همان طلا تا خروج:

```
h = min(Manhattan(agent, gold) + Manhattan(gold, exit))
```

بعد از گرفتن طلا:

```
h = Manhattan(agent, exit)
```

پیاده‌سازی واقعی:

```

def _heuristic(self, state: SearchState) -> int:
    if state.has_gold:
        return self._manhattan(state.position, self.config.exit_position)
    return min(
        self._manhattan(state.position, gold)
        + self._manhattan(gold, self.config.exit_position)
        for gold in self.gold_positions
    )

```

این heuristic دیوارها، غول‌ها و خسارت چاه را نادیده می‌گیرد؛ بنابراین یک تخمین خوش‌بینانه و lower bound از هزینه باقی‌مانده است.

۴.۵ صف اولویت و بازسازی مسیر

برای frontier از `heapq` استفاده شده است. هر عضو heap شامل اولویت، هزینه فعلی، شمارنده حل تساوی و حالت است:

```

frontier: list[tuple[int, int, int, SearchState]] = []
heappush(frontier, (self._heuristic(start_state), 0, next(tie_breaker), start_state))

```

دیکشنری `came_from` والد هر حالت و اکشن منتهی به آن را ذخیره می‌کند. پس از رسیدن به هدف، مسیر و لیست اکشن‌ها از انتها به ابتدا بازسازی و سپس معکوس می‌شوند.

۴.۶ نحوه استفاده در پروژه

عامل A-Star هنگام `reset()` یک برنامه کامل از شروع تا طلا و خروج محاسبه می‌کند. سپس `choose_action()` حرکت‌های ذخیره‌شده را یکی‌یکی برمی‌گرداند. اگر موقعیت واقعی با موقعیت مورد انتظار متفاوت باشد، عامل از وضعیت فعلی دوباره برنامه‌ریزی می‌کند.

۴.۷ مزایا و محدودیت‌ها

مزایا:

- مسیر قابل تحلیل و تکرارپذیر؛
- در نظر گرفتن جان و هزینه چاه؛
- موفقیت بسیار بالا در محیط دارای مسیر امن؛
- مناسب برای baseline و ارزیابی کران بالای عملکرد.

محدودیت اصلی:

A-Star به کل نقشه دسترسی دارد. بنابراین نتیجه آن را نباید به یک عامل واقعی در محیط ناشناخته تعمیم داد.

۵. روش دوم: پایگاه دانش و عامل Rule-Based

۵.۱ ایده اصلی

عامل Rule-Based نقشه را مشاهده نمی‌کند. این عامل با استفاده از ادراک‌های محلی، مجموعه‌ای از اطلاعات درباره خانه‌ها ایجاد می‌کند. این اطلاعات در کلاس `KnowledgeBase` ذخیره می‌شوند.

مهم‌ترین مجموعه‌های پایگاه دانش عبارت‌اند از:

مجموعه	معنی
<code>visited</code>	خانه‌های بازدید شده
<code>safe</code>	خانه‌های اثبات شده امن
<code>walls</code>	دیوارهای مشاهده شده
<code>no_pit</code>	خانه‌هایی که چاه نیستند
<code>no_wumpus</code>	خانه‌هایی که غول نیستند
<code>possible_pits</code>	خانه‌های مشکوک به چاه
<code>possible_wumpus</code>	خانه‌های مشکوک به غول
<code>definite_pits</code>	چاه‌های قطعی
<code>definite_wumpus</code>	محل قطعی غول

۵.۲ قواعد استنتاج

نبود Breeze

اگر در خانه فعلی Breeze وجود نداشته باشد، هیچ‌کدام از همسایه‌های قابل حرکت آن چاه نیستند:

```
-Breeze(x) → x: های مجاور y برای تمام → -Pit(y)
```

نبود Stench

اگر Stench وجود نداشته باشد، همسایه‌های قابل حرکت گول ندارند:

```
-Stench(x) → x: های مجاور y برای تمام → -Wumpus(y)
```

وجود Breeze یا Stench

وجود Breeze به معنی آن است که حداقل یکی از همسایه‌ها ممکن است چاه باشد. این همسایه‌ها به‌صورت یک clause ذخیره می‌شوند. برای Stench نیز همین منطق برای گول اجرا می‌شود.

اگر پس از حذف خانه‌های مبرا و دیوارها، یک clause فقط یک عضو داشته باشد، آن عضو به‌عنوان خطر قطعی ثبت می‌شود:

```
if len(candidates) == 1:
    only = next(iter(candidates))
    self.definite_pits.add(only)
```

ثبت چاه تجربه‌شده

اگر عامل وارد چاه شود و زنده بماند، observation مقدار `pit_here=True` دارد. پایگاه دانش این خانه را مستقیماً به‌عنوان `DEFINITE_PIT` ثبت می‌کند و آن را از مجموعه امن حذف می‌کند. این موضوع از تناقض میان تجربه عامل و حافظه آن جلوگیری می‌کند.

۵.۳ محاسبه ریسک

وقتی هیچ خانه کاملاً امنی باقی نماند، عامل مجبور است یک خانه ناشناخته را انتخاب کند. برای این کار، تابع `risk()` استفاده می‌شود:

```

risk = 1.0
risk += 5.0 * self.evidence_wumpus.get(position, 0)
risk += 1.5 * self.evidence_pit.get(position, 0)
if position in self.definite_pits:
    risk += 10.0
if position in self.visited:
    risk -= 0.25

```

- دیوار و غول قطعی دارای ریسک بی‌نهایت‌اند.
- خانه امن ریسک صفر دارد.
- شواهد غول ضریب بیشتری از شواهد چاه دارند، زیرا غول مرگ فوری ایجاد می‌کند.
- چاه قطعی بسیار پرریسک است، ولی به دلیل امکان زنده ماندن کاملاً ممنوع نشده است.

۵.۴ ترتیب تصمیم‌گیری عامل

تابع `choose_action()` تصمیم را در چهار مرحله می‌گیرد:

1. **بازگشت پس از گرفتن طلا:** پیدا کردن کوتاه‌ترین مسیر امن شناخته‌شده به خروج؛
2. **اکتشاف امن:** حرکت به نزدیک‌ترین خانه امن و بازدیدنشده؛
3. **انتخاب frontier کم‌خطر:** وقتی خانه امن جدیدی باقی نمانده است؛
4. **Fallback:** انتخاب کم‌خطرترین حرکت محلی معتبر.

flowchart TD

```

A[observation دریافت] --> B[KnowledgeBase به روزرسانی]
B --> C[طلا گرفته شده؟]
C -- بله --> D[مسیر امن به خروج وجود دارد؟]
C -- خیر --> F[ادامه بررسی گزینه‌ها]
D -- بله --> E[حرکت در مسیر امن خروج]
D -- خیر --> F
F --> G[خانه امن بازدیدنشده وجود دارد؟]
G -- بله --> H[امن backtracking و BFS]
G -- خیر --> I[قابل دسترس وجود دارد؟ frontier]
H --> J[risk با کمترین frontier انتخاب]
I -- بله --> J
I -- خیر --> K[کم‌خطرترین حرکت محلی]

```

۵.۵ Backtracking

عامل برای رسیدن به یک خانه امن جدید، الزاماً مستقیم به آن حرکت نمی‌کند. ممکن است ابتدا از خانه‌های امن قبلی برگردد. این مسیر با BFS روی مجموعه خانه‌های امن ساخته می‌شود.

صف BFS با `deque` پیاده‌سازی شده و دیکشنری `parent` برای بازسازی مسیر به کار می‌رود. این همان بخش backtracking عامل است: وقتی شاخه فعلی اکتشاف پایان می‌یابد، عامل بدون ورود به خطر شناخته‌شده، از مسیر امن قبلی به frontier دیگری می‌رود.

۵.۶ قابلیت توضیح تصمیم

هر تصمیم در DecisionTrace ذخیره می‌شود:

- ادراک‌های فعلی؛
- استنتاج‌های انجام‌شده؛
- گزینه‌های بررسی‌شده؛
- حرکت نهایی و دلیل انتخاب آن.

این قابلیت باعث می‌شود عامل Rule-Based برای ارائه و دفاع بسیار توضیح‌پذیر باشد.

۶. الگوریتم ژنتیک برای آموزش سیاست حرکت

۶.۱ هدف استفاده از GA

در عامل قاعده‌محور، اولویت‌ها و ضرایب ریسک به صورت دستی تعیین می‌شوند. در روش سوم، هدف این است که اهمیت ویژگی‌های مختلف حرکت به صورت خودکار و بر اساس عملکرد عامل روی مجموعه‌ای از نقشه‌های آموزشی تنظیم شود. الگوریتم ژنتیک مستقیماً مسیر ثابت تولید نمی‌کند. خروجی آن مجموعه‌ای از ۱۰ وزن است که در زمان اجرای عامل برای امتیازدهی حرکت‌ها استفاده می‌شوند.

۶.۲ نمایش کروموزوم

هر کروموزوم یک بردار ده‌بعدی از اعداد حقیقی است:

نقش	ژن
پاداش ورود به خانه امن	safe_bonus
پاداش ورود به خانه بازدیدنشده	unvisited_bonus
اهمیت نزدیک شدن به خروج پس از طلا	exit_progress_weight
جریمه شواهد چاه	pit_risk_penalty
جریمه شواهد گول	wumpus_risk_penalty
تمایل یا عدم تمایل به خانه ناشناخته	unknown_weight
جریمه بازدید تکراری	revisit_penalty
جریمه برگشت فوری به خانه قبلی	reverse_penalty
پاداش دسترسی به ناحیه کشف‌نشده	frontier_bonus
احتیاط بیشتر در جان پایین	health_caution_penalty

محدوده هر ژن در GENE_BOUNDS تعریف شده است تا crossover و mutation مقدار نامعتبر تولید نکنند.

۶.۳ تابع Fitness

هر کروموزوم روی تمام ۱۲ نقشه آموزشی اجرا می‌شود و Fitness نهایی برابر میانگین Fitness اپیزودهاست. فرمول واقعی تابع `episode_fitness()` به شکل زیر است:

```
value = 1500.0 if success else 0.0
value += 250.0 * collected_gold
value += 2.0 * remaining_health
value -= 2.0 * steps
value -= 180.0 * pit_entries
terminal_penalties = {
    "wumpus": -1400.0,
    "health_depleted": -800.0,
    "escaped_without_gold": -700.0,
    "max_steps": -400.0,
    "agent_stopped": -500.0,
}
value += terminal_penalties.get(termination_reason, 0.0)
```

هدف این تابع ایجاد تعادل میان موارد زیر است:

- موفقیت و جمع‌آوری طلا؛
- حفظ جان؛
- کاهش تعداد حرکت؛
- اجتناب از چاه؛
- جلوگیری شدید از مرگ با گول؛
- جلوگیری از توقف یا خروج بدون طلا.

Fitness مستقل از score محیط محاسبه می‌شود تا یک معیار دوبار شمرده نشود.

۶.۴ جمعیت اولیه

جمعیت شامل ۲۴ کروموزوم است. یک عضو جمعیت از وزن‌های دستی اولیه ساخته می‌شود و بقیه اعضا به صورت تصادفی و در محدوده مجاز ژن‌ها تولید می‌شوند:

```
population: list[Genome] = [GeneticWeights().as_genome()]
while len(population) < self.population_size:
    population.append(
        [self.rng.uniform(*GENE_BOUNDS[name]) for name in GENE_NAMES]
    )
```

۶.۵ Selection

از **Tournament Selection** استفاده شده است. در هر انتخاب، سه عضو تصادفی از جمعیت انتخاب می‌شوند و عضوی که Fitness بیشتری دارد به عنوان والد برگزیده می‌شود:

```
indexes = self.rng.sample(range(len(population)), self.tournament_size)
winner = max(indexes, key=lambda index: fitnesses[index])
```

این روش فشار انتخاب مناسبی ایجاد می‌کند، ولی همچنان به اعضای مختلف جمعیت فرصت والد شدن می‌دهد.

۶.۶ Crossover

Crossover از نوع arithmetic/intermediate است. برای هر ژن، ضریب α به صورت تصادفی انتخاب شده و دو فرزند میان دو والد ساخته می‌شوند:

```
child1.append(alpha * value1 + (1.0 - alpha) * value2)
child2.append((1.0 - alpha) * value1 + alpha * value2)
```

احتمال انجام crossover برابر 0.90 است. اگر crossover انجام نشود، والد‌ها بدون ترکیب کپی می‌شوند.

۶.۷ Mutation

هر ژن با احتمال 0.10 یک تغییر تصادفی گاوسی دریافت می‌کند:

```
mutated[index] += self.rng.gauss(0.0, self.mutation_sigma)
```

مقدار mutation_sigma برابر 2.0 است. بعد از جهش، تمام ژن‌ها در محدوده تعریف شده clip می‌شوند.

۶.۸ Early Stopping و Elitism

در هر نسل، دو کروموزوم برتر بدون تغییر به نسل بعد منتقل می‌شوند. این کار مانع از دست رفتن بهترین جواب پیداشده می‌شود.

اگر بهترین Fitness برای ۸ نسل بهتر نشود، آموزش می‌تواند زودتر متوقف شود. در اجرای ذخیره شده، آموزش تمام ۲۴ نسل را اجرا کرده است.

۶.۹ پارامترهای واقعی آموزش

پارامتر	مقدار
تعداد نقشه‌های آموزش	12
Population size	24
Maximum generations	24
Crossover rate	0.90
Mutation rate	0.10
Mutation sigma	2.0
Elite count	2

پارامتر	مقدار
Tournament size	3
Patience	8
Max steps	250
Seed	17
Best fitness	1840.6667

۶.۱۰ جریان آموزش

```
flowchart TD
    A[اجرای هر کروموزوم روی 12 نقشه] --> B[تولید جمعیت اولیه]
    B --> C[Fitness محاسبه می‌نکین]
    C --> D[مرتب‌سازی جمعیت]
    D --> E[Elitism ذخیره بهترین اعضا با]
    E --> F[Tournament Selection]
    F --> G[Arithmetic Crossover]
    G --> H[Gaussian Mutation]
    H --> I[ژن‌ها در محدوده مجاز Clip]
    I --> J{Early Stop؟ نسل آخر یا}
    J -- خیر --> B
    J -- بله --> K[best_weights.json ذخیره]
```

۶.۱۱ وزن‌های نهایی

وزن‌های تکامل‌یافته ذخیره‌شده در `best_weights.json` عبارت‌اند از:

وزن نهایی	ژن
10.6256	safe_bonus
11.6390	unvisited_bonus
22.9999	exit_progress_weight
10.5239-	pit_risk_penalty
21.0707-	wumpus_risk_penalty
9.3814-	unknown_weight
8.7879-	revisit_penalty
9.9740-	reverse_penalty
2.4071	frontier_bonus
15.1342-	health_caution_penalty

بزرگ بودن `exit_progress_weight` نشان می‌دهد حرکت مؤثر به سمت خروج اهمیت زیادی دارد. مقادیر منفی بزرگ برای `wumpus_risk_penalty`، `unknown_weight` و `reverse_penalty` نیز نشان می‌دهند سیاست آموزش‌دیده از خطر غول، خانه ناشناخته و برگشت فوری اجتناب می‌کند.

۷. عامل Hybrid Genetic

۷.۱ دلیل نام‌گذاری «ترکیبی»

عامل سوم فقط یک سیاست خطی ژنتیکی نیست. این عامل سه جزء را ترکیب می‌کند:

۱. پایگاه دانش محلی برای استخراج شواهد خطر؛
۲. وزن‌های تکامل‌یافته برای امتیازدهی حرکت‌ها در مرحله اکتشاف؛
۳. بازگشت deterministic از کوتاه‌ترین مسیر امن پس از گرفتن طلا.

به همین دلیل، نام دقیق روش **Hybrid Genetic Agent** است.

۷.۲ استخراج ویژگی حرکت

عامل برای هر حرکت معتبر، خانه مقصد را پیدا کرده و ده ویژگی تولید می‌کند. نمونه‌ای از ویژگی‌ها:

- آیا خانه مقصد امن است؟
- آیا قبلاً دیده نشده است؟
- چند بار بازدید شده است؟
- آیا برگشت فوری به خانه قبلی است؟
- میزان شواهد چاه و غول چقدر است؟
- چند خانه کشف‌نشده در اطراف مقصد وجود دارد؟
- در جان پایین، ورود به این خانه چقدر نامطمئن است؟

۷.۳ فرمول تصمیم

امتیاز هر حرکت برابر ضرب داخلی بردار ویژگی‌ها در وزن‌هاست:

```
score(action) = \sum weight_i \times feature_i(action)
```

پیاده‌سازی واقعی:

```
def _weighted_score(self, features: dict[str, float]) -> float:
    return sum(
        float(getattr(self.weights, name)) * float(features[name])
        for name in GENE_NAMES
    )
```

حرکتی که بیشترین امتیاز را داشته باشد انتخاب می‌شود. در صورت تساوی دقیق، ترتیب ثابت زیر tie را می‌شکنند و اجرای پروژه را تکرارپذیر نگه می‌دارد:

```
RIGHT, DOWN, LEFT, UP
```

۷.۴ جلوگیری از خروج زودهنگام

اگر عامل هنوز طلا نداشته باشد، حرکت به خروج امتیاز بسیار منفی می‌گیرد:

```
if target == self.exit_position and not has_gold:
    scored.append(
        (-1_000_000.0, order_index, action, {"premature_exit": 1.0})
    )
```

این محدودیت مانع پایان ناموفق زودهنگام می‌شود.

۷.۵ بازگشت پس از طلا

بعد از گرفتن طلا، اگر مسیر امن شناخته‌شده‌ای به خروج وجود داشته باشد، عامل دیگر صرفاً greedy بر اساس امتیاز حرکت عمل نمی‌کند. با BFS کوتاه‌ترین مسیر شناخته‌شده امن را پیدا کرده و آن را دنبال می‌کند. این بخش باعث افزایش قابلیت اطمینان عامل در مرحله بازگشت می‌شود. در نتیجه الگوریتم ژنتیک بیشتر مسئول تعیین رفتار اکتشافی و ریسک‌پذیری است، نه تمام مراحل اپیزود.

۷.۶ قابلیت Trace

عامل ژنتیکی برای هر حرکت امتیاز تمام کاندیدها را در GeneticDecisionTrace ذخیره می‌کند. بنابراین در اجرای verbose می‌توان مشاهده کرد هر ویژگی چه مقداری داشته و چرا یک حرکت انتخاب شده است.

۸. تولید نقشه و طراحی آموزش و آزمایش

۸.۱ فرمت نقشه

فایل نقشه دقیقاً شامل ۱۲ خط است:

1. هشت خط اول: گرید 8×8؛
2. خط نهم: جان اولیه؛
3. خط دهم: امتیاز طلا؛
4. خط یازدهم: جریمه چاه؛
5. خط دوازدهم: مختصات خروج به صورت one-based.

نمونه maps/sample_01.txt:


```
*****
P*D***P*
**D*****
*D*D****
P*DD**G*
W***DD**
D*****
*DPD*W**
120
50
10
8 8
```

در این نقشه، جان اولیه ۱۲۰، امتیاز هر طلا ۵۰، جریمه هر بار ورود به چاه ۱۰ و خروج در خانه (8,8) قرار دارد.

۸.۲ اعتبارسنجی

`map_parser.py` موارد زیر را بررسی می‌کند:

- تعداد دقیق خطوط؛
- طول دقیق هر سطر؛
- مجاز بودن نمادها؛
- مثبت بودن جان اولیه؛
- منفی نبودن امتیاز طلا و جریمه چاه؛
- امن بودن خانه شروع؛
- امن و معتبر بودن خروج؛
- وجود حداقل یک طلا.

۸.۳ مجموعه آموزش

برای GA از ۱۲ نقشه استفاده شده است:

- ۴ نقشه آسان؛
- ۴ نقشه متوسط؛
- ۴ نقشه سخت.

Seed خود الگوریتم ژنتیک 17 است. نقشه‌های آموزش مستقل از نقشه‌های تست هستند.

۸.۴ مجموعه تست

ارزیابی نهایی روی ۳۰ نقشه دیده‌نشده انجام شده است:

- ۱۰ آسان؛
- ۱۰ متوسط؛
- ۱۰ سخت.

Seed تولید مجموعه تست برابر 20260730 است. تمام نقشه‌ها جان اولیه ۱۲۰ دارند تا مقایسه امتیاز میان سطوح‌های دشواری تحت تأثیر جان اولیه متفاوت نباشد.

مولد نقشه حداقل یک مسیر امن از شروع به طلا و سپس خروج ایجاد می‌کند، ولی محل خروج، طلا، موانع و طول مسیر میان نقشه‌ها متنوع هستند.

۹. طراحی آزمایش نهایی

هر سه عامل روی دقیقاً همان ۳۰ نقشه اجرا شده‌اند. سقف حرکت هر اپیزود ۲۵۰ است. معیارهای اصلی عبارت‌اند از:

- موفقیت یا شکست؛
- امتیاز نهایی؛
- جان باقی‌مانده؛
- تعداد حرکت؛
- تعداد ورود به چاه؛
- علت پایان؛
- زمان اجرا؛
- تعداد گره‌های گسترش‌یافته برای A-Star.

زمان اجرا با میانه سه اجرای کامل اندازه‌گیری شده است تا تأثیر نوسانات کوتاه‌مدت سیستم کمتر شود.

دو معیار جدا برای حرکت گزارش شده است:

- `average_steps_all`: میانگین حرکت تمام اپیزودها؛
- `average_steps_success`: میانگین حرکت فقط در اپیزودهای موفق.

این جداسازی مهم است؛ زیرا شکست سریع با غول نباید به اشتباه به‌عنوان مسیر کوتاه و خوب تفسیر شود.

۱۰. نتایج آزمایش

۱۰.۱ نتایج کلی

روش	اجرا	موفقیت	نرخ موفقیت	امتیاز متوسط همه	حرکت متوسط همه	حرکت متوسط موفق	جان متوسط	چاه متوسط	مرگ غول
A-Star	30	30	100.00%	157.60	12.40	12.40	107.60	0.000	0
-Rule Based	30	27	90.00%	117.93	32.90	32.30	73.93	0.267	1

روش	اجرا	موفقیت	نرخ موفقیت	امتیاز متوسط همه	حرکت متوسط همه	حرکت متوسط موفق	جان متوسط	چاه متوسط	مرگ غول
Hybrid Genetic	30	25	83.33%	120.97	31.80	24.60	75.63	0.133	2

۱۰.۲ نتایج بر اساس دشواری

روش	آسان	متوسط	سخت
A-Star	100%	100%	100%
Rule-Based	90%	100%	80%
Hybrid Genetic	100%	70%	80%

۱۰.۳ تحلیل A-Star

A-Star در هر ۳۰ نقشه موفق شد و میانگین مسیر آن ۱۲.۴ حرکت بود. این نتیجه به دلیل دو عامل قابل انتظار است:

- ۱. مولد نقشه وجود حداقل یک مسیر امن را تضمین می‌کند؛
- ۲. A-Star کل محل دیوارها، چاه‌ها، غول‌ها، طلا و خروج را می‌داند.

بنابراین عملکرد آن کران بالای مجموعه آزمایش است، نه یک نتیجه قابل تعمیم به محیط ناشناخته.

۱۰.۴ تحلیل Rule-Based

عامل قاعده‌محور در ۲۷ نقشه از ۳۰ نقشه موفق شد و بهترین نرخ موفقیت را میان دو عامل آنلاین داشت. این عامل محتاط‌تر است و معمولاً قبل از ورود به ناحیه جدید، خانه‌های امن را بررسی و backtracking می‌کند. این احتیاط باعث افزایش تعداد حرکت می‌شود، ولی احتمال مرگ را کاهش می‌دهد.

میانگین حرکت موفق آن ۳۲.۳۰ است که بیشتر از عامل ژنتیکی است. در مقابل، فقط یک مرگ با غول ثبت کرده است.

۱۰.۵ تحلیل Hybrid Genetic

عامل ژنتیکی ترکیبی در ۲۵ نقشه موفق شد. نرخ موفقیت آن کمتر از Rule-Based بود، اما ویژگی‌های زیر را نشان داد:

- امتیاز متوسط همه اجراها کمی بالاتر از Rule-Based است؛
- در اجراهای موفق، میانگین حرکت ۲۴.۶۰ در برابر ۳۲.۳۰ است؛
- میانگین ورود به چاه کمتر است؛
- تعداد مرگ با غول بیشتر است.

این نتیجه نشان می‌دهد سیاست تکامل‌یافته در seed فعلی، رفتار تهاجمی‌تری پیدا کرده است. این سیاست هنگام موفقیت سریع‌تر به نتیجه می‌رسد، ولی در بعضی شرایط مبهم ریسک بیشتری می‌پذیرد.

۱۰.۶ جمع‌بندی مقایسه

اولویت	روش مناسب‌تر
داشتن نقشه کامل و مسیر کم‌هزینه	A-Star
توضیح‌پذیری و نرخ موفقیت آنلاین بیشتر	Rule-Based
مسیر کوتاه‌تر در موفقیت‌ها و سیاست قابل آموزش	Hybrid Genetic

هیچ‌یک از دو عامل آنلاین در همه معیارها برنده مطلق نیست. انتخاب روش به اولویت سیستم بستگی دارد: قابلیت اعتماد و توضیح‌پذیری یا سرعت اکتشاف و کوتاهی مسیر موفق.

۱۱. تست و قابلیت بازتولید

پروژه دارای ۴۴ تست خودکار است که بخش‌های اصلی زیر را پوشش می‌دهند:

- قوانین محیط و مرزهای گرید؛
- دیوار، چاه، غول، طلا و خروج؛
- parser و نقشه نامعتبر؛
- مسیر A-Star و انتخاب مسیر امن‌تر؛
- قواعد Knowledge Base؛
- backtracking عامل Rule-Based؛
- خواندن و ذخیره وزن‌های ژنتیکی؛
- اجرای آموزش کوچک GA؛
- مولد نقشه و تکرارپذیری seed؛
- خلاصه‌سازی نتایج و benchmark.

خروجی نهایی اجرای تست‌ها:

44 passed

همچنین تمام فایل‌های Python با دستور زیر بدون خطای نحوی بررسی می‌شوند:

```
python -m compileall -q .
```

Seedهای ثابت، ترتیب ثابت حرکت‌ها، نقشه‌های ذخیره‌شده، `best_weights.json` و CSV نتایج، امکان بازتولید خروجی‌های اصلی را فراهم می‌کنند.

۱۲. محدودیت‌های علمی

1. **برتری اطلاعاتی A-Star:** این عامل کل نقشه را می‌بیند؛ بنابراین مقایسه مستقیم آن با عامل‌های آنلاین باید با احتیاط انجام شود.
2. **وابستگی GA به داده آموزش:** وزن‌های نهایی تحت تأثیر ۱۲ نقشه آموزشی، تابع Fitness و seed هستند.
3. **عدم تضمین موفقیت:** عامل ژنتیکی و Rule-Based در محیط ناشناخته تضمین ریاضی برای موفقیت ندارند.
4. **محدود بودن استنتاج:** پایگاه دانش از قواعد مستقیم و clause‌های محلی استفاده می‌کند و یک SAT solver کامل نیست.
5. **یک seed اصلی تست:** برای نتیجه آماری قوی‌تر باید چند seed مستقل و فاصله اطمینان گزارش شود.
6. **وابستگی runtime به سخت‌افزار:** زمان‌های اجرا به سیستم، نسخه Python و بار پردازشی وابسته‌اند.
7. **غول غیرقابل حذف است:** در این مدل تیر و عمل شلیک وجود ندارد و غول تا پایان ثابت باقی می‌ماند.
8. **گرید ثابت:** اندازه محیط در نسخه فعلی 8×8 است.

۱۳. نتیجه‌گیری

در این پروژه سه دیدگاه مهم هوش مصنوعی در یک محیط مشترک پیاده‌سازی و مقایسه شدند. A-Star نشان داد وقتی اطلاعات کامل در دسترس باشد، جست‌وجوی آگاهانه می‌تواند مسیر بسیار قابل اعتماد و کم‌هزینه‌ای پیدا کند. عامل Rule-Based نشان داد با استفاده از ادراک‌های محلی، قواعد منطقی، حافظه و backtracking می‌توان بدون مشاهده نقشه به نرخ موفقیت بالایی رسید. عامل Hybrid Genetic نیز نشان داد الگوریتم ژنتیک می‌تواند ضرایب یک سیاست تصمیم‌گیری را به‌طور خودکار تنظیم کند و در اجراهای موفق مسیرهای کوتاه‌تری نسبت به قواعد دستی ایجاد کند.

نتیجه نهایی این است که:

- برای محیط کاملاً شناخته‌شده، A-Star بهترین گزینه است؛
 - برای محیط ناشناخته‌ای که قابلیت توضیح و اطمینان اهمیت دارد، Rule-Based مناسب‌تر است؛
 - برای محیطی که امکان آموزش وجود دارد و کوتاهی مسیر موفق اهمیت بیشتری دارد، Hybrid Genetic انتخاب قابل‌بررسی است.
- پروژه یک pipeline کامل شامل تولید نقشه، آموزش، اجرای عامل‌ها، ارزیابی، تست، ذخیره نتایج و تولید گزارش فراهم می‌کند.

۱۴. پیشنهادهای توسعه آینده

- اجرای آزمایش روی چند seed و گزارش میانگین و confidence interval؛
- افزایش تعداد و تنوع نقشه‌های آموزش و تست؛
- استفاده از استنتاج SAT یا منطق احتمالاتی در Knowledge Base؛
- استفاده از multi-objective GA برای بهینه‌سازی جداگانه موفقیت، حرکت و ریسک؛
- افزودن قابلیت شلیک تیر و تغییر وضعیت غول؛

- مقایسه با روش‌های Reinforcement Learning؛
- پشتیبانی از اندازه‌های مختلف گرید؛
- افزودن رابط گرافیکی برای نمایش تصمیم‌ها و دانش عامل.

۱۵. راهنمای اجرای پروژه

ساخت محیط مجازی

```
python -m venv .venv
```

فعال‌سازی در Windows:

```
.venv\Scripts\activate
```

فعال‌سازی در Linux یا macOS:

```
source .venv/bin/activate
```

نصب وابستگی‌ها

```
pip install -r requirements.txt
```

اجرای تست‌ها

```
pytest -q
```

اجرای نمایشی هر سه عامل

```
python demo_all.py --map maps/sample_01.txt
```

اجرای جداگانه عامل‌ها

```
python main.py --agent astar --map maps/sample_01.txt  
python main.py --agent rule --map maps/sample_rule_reasoning.txt --max-steps 250  
python main.py --agent genetic --map maps/sample_01.txt --max-steps 250
```

```
python train_genetic.py \  
  --population 24 \  
  --generations 24 \  
  --patience 8 \  
  --max-steps 250 \  
  --seed 17
```

اجرای آزمایش نهایی

```
python experiment.py
```

استفاده مجدد از نقشه‌های تست موجود

```
python experiment.py --skip-generate
```

بررسی کامل بسته تحویل

```
python verify_delivery.py
```

ساخت مجدد گزارش PDF و اسلایدها

```
pip install -r requirements-docs.txt  
python docs/build_artifacts.py
```

۱۶. منابع

۱. Russell, S. J., & Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson.